

Deep Learning And Its Application Assignment

line 1: Muhammad Haider Abbas
line 2: Master's of Computer Science and Technology
line 3: Ulster University Birmingham Campus
line 4: Birmingham, West Midlands, United Kingdom
line 5: Abbas-MH@ulster.ac.uk

Abstract: In this paper, we evaluate how a Q-learning agent can learn to navigate a 5×5 grid world containing obstacles, sparse rewards, and the ability to jump +5 spaces as an addition to standard Q-learning methods. The agent used an ϵ -greedy exploration strategy to find an optimal path based on trial and error with respect to balancing exploration against exploitation. We evaluated four different ϵ value settings (0.1, 0.3, 0.5 and 0.9) in order to assess differences in learning efficiency and convergence to the optimal policy for each ϵ value used. Results from our experimentation indicated that moderate levels of exploration (values between 0.1 and 0.3) resulted in the highest levels of performance (cumulative reward greater than 10) within the 30-40 episode range while providing the agent with opportunities to avoid obstacles and obtain reward opportunities in order to reach its goal; thus demonstrating how the effective tuning of parameters affects Q-learning agent performance.

Keywords: Reinforcement Learning, Q-Learning, Grid World, Exploration-Exploitation, Epsilon-Greedy, Bellman Equation

I. INTRODUCTION (HEADING 1)

An approach to machine learning called Reinforcement Learning (RL) is concerned with teaching an autonomous agent to optimise its choices through repeated association with its environment. The agent interacts with the environment through actions and receives feedback in the form of rewards/penalty feedback to refine its decision-making policy for maximising its cumulative received rewards [1]. Q-learning, an algorithm introduced by Watkins [2], is one of the most widely used algorithms currently because Q-learning uses an off-policy learning method and has convergence requirements based on tabular representation of data.

This work investigates a deterministic 5×5 grid-world environment containing obstacles, sparse rewards, and a jump transition. The objective is for the agent to navigate from the start state to the terminal state while maximising cumulative reward.

The objectives are to formulate the environment, implement a Q-learning agent, evaluate ϵ -greedy exploration strategies, and analyse learned policies.

II. GRID WORLD PROBLEM FORMATION

A. Environment Configuration (Heading 2)

The environment is a deterministic, fully observable 5×5 grid bounded by borders. Using zero-indexed coordinates for computational convenience (converted from the 1-indexed problem specification), we define:

- **State space:** All valid (row, column) pairs where row, col $\in \{0, 1, 2, 3, 4\}$.
- **Initial state:** (1,0) corresponding to 1-indexed [2,1].

- **Terminal state:** (4,4) corresponding to [5,5] with reward +10.
- **Action space:** Four discrete actions – North (0), South (1), East (2), West (3).
- **Obstacles:** Cells that are impassable. Based on typical grid configurations, obstacles are placed at positions (1,1), (1,2), (2,1), (3,1) in zero-indexing.
- **Special transition:** From state (1,3) ([2,4] in 1-index) to state (3,3) ([4,4]) with an immediate reward of +5.
- **Step penalty:** All actions that result in valid non-terminal, non-jump transitions yield reward -1. Attempted moves into obstacles or out of bounds keep the agent in its current cell and still incur the -1 penalty.

Parameter	Value
Grid size	5×5
Total cells	25
Valid states (excluding obstacles)	21
Actions	North, South, East, West
Start state (1-indexed)	[2,1]
Start state (0-indexed)	(1,0)
Terminal state	[5,5] / (4,4)
Terminal reward	+10
Jump source	[2,4] / (1,3)
Jump destination	[4,4] / (3,3)
Jump reward	+5
Step penalty	-1
Obstacle cells (0-indexed)	(1,1), (1,2), (2,1), (3,1)

a. Table I. Environment Configuration Summary

B. State Transition Dynamics

Let $s = (r, c)$ denote the current state and $a \in \{N, S, E, W\}$ the chosen action. The next state s' and reward $R(s, a, s')$ are determined as follows:

1. Compute candidate $s_cand = (r-1, c)$ for N, $(r+1, c)$ for S, $(r, c+1)$ for E, $(r, c-1)$ for W.
2. If s_cand is outside the $[0,4]$ bounds or corresponds to an obstacle cell, $s' = s$ and $R = -1$.
3. Else if s_cand equals the jump source (1,3), then $s' = (3,3)$ and $R = +5$.

4. Else if s_{cand} equals the terminal (4,4), then $s' = (4,4)$, $R = +10$, and the episode terminates.
5. Otherwise, $s' = s_{\text{cand}}$ and $R = -1$.

This formulation captures all requirements from the problem specification.

III. THEORETICAL FRAMEWORK

A. Exploration Vs Exploitation

Reinforcement learning faces the exploration–exploitation dilemma: whether to choose actions that maximise current estimated reward (exploitation) or to explore uncertain actions that may yield greater long-term benefit.

The ϵ -greedy strategy addresses this trade-off by selecting a random action with probability ϵ and the greedy action with probability $1-\epsilon$. Proper tuning of ϵ ensures sufficient exploration while allowing convergence toward an optimal policy in deterministic environments.

B. Bellman Equation and Q-Learning Update

The Bellman optimality equation expresses the relationship between the value of a state-action pair and the values of subsequent states under an optimal policy. For Q-learning, the update rule is:

$$Q(s, a) \leftarrow Q(s, a) + [r + \max_{a'} Q(s', a') - Q(s, a)] \quad (1)$$

This equation underpins the learning process. Each term plays a critical role:

- **Q (s, a):** Current estimate of the expected cumulative reward when taking action a in state s.
- **α (learning rate):** Determines how much new information overrides old information. $\alpha = 1$ discards prior estimates entirely, while smaller values perform smoothing.
- **γ (discount factor):** Balances immediate versus future rewards. γ close to 1 makes the agent far-sighted.
- **$r + \gamma \max_{a'} Q(s', a')$:** The Temporal Difference (TD) target – the immediate reward plus the discounted best future value.
- **$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$:** The TD error, representing the surprise or new information

Unlike SARSA, which updates values based on the action actually taken, Q-learning uses the maximum possible next-state value. This makes it an off-policy method capable of converging to the optimal policy independently of the behavior policy.

IV. Q-TABLE CONSTRUCTION AND AGENT DESIGN

A. Q-Table Structure

We implemented the Q-table as a dictionary that maps pairs of states (row, column) to numpy arrays with 4 elements, each corresponding to the Q-value for one of the 4 actions. All of the initial values were set to zero as we did not want to bias the results using optimistic initialisation. If we

have a 5x5 grid with k obstacles, there are $(25 - |\text{obstacles}|)$ valid states (assuming 4 obstacles) to learn for Q-Learning; hence, there are 21 states and so $21 \times 4 = 84$ different Q-values that need to be learned.

We have tried a number of different learning rates for Q-learning: $\alpha = .1, .3, .5, .7$ and 1.0. Different learning rates (0.1–1.0) were tested to evaluate convergence stability.

Learning Rate	Convergence Speed	Final Avg Reward	Stability
0.1	Moderate	9.33	High
0.3	Moderate-Fast	8.91	High
0.5	Fast	7.84	Medium
0.7	Fast	6.52	Low
1.0	Very Fast	5.13	Very Poor (oscillating)

b. Table II. Learning Rate (a) Comparison Results

B. Agent Implementation

The Q-learning agent class encapsulates:

1. **Action selection:** The `get_action(state, training)` method implements ϵ -greedy. During training, random actions occur with probability ϵ ; during evaluation, only exploitation is used ($\epsilon = 0$).
2. **Experience update:** The `update(state, action, reward, next_state, done)` method applies equation (1). When `done` is True (terminal state reached), the $\max_{a'} Q(s', a')$ term is omitted because no future states exist.
3. **Epsilon decay:** To balance early exploration with late exploitation, ϵ is multiplied by a decay factor (0.995) after each episode, with a minimum floor of 0.01. This ensures the agent explores widely at the beginning but settles into greedy behaviour once a good policy is discovered.
4. **Policy extraction:** The greedy policy is derived by selecting, for each state, the action that maximises $Q(s, a)$.

Hyperparameter	Value	Justification
Learning rate (α)	0.1	Balances convergence speed and stability
Discount factor (γ)	0.95	Favours long-term rewards over immediate
Initial epsilon (ϵ)	0.1	10% random actions for sufficient exploration
Epsilon decay	0.995	Gradual reduction per episode

Hyperparameter	Value	Justification
Minimum epsilon	0.01	Maintains occasional exploration
Maximum episodes	100	As per specification
Early stop threshold	Avg reward > 10	As per specification
Early stop window	30 episodes	As per specification

c. Table III. HYPERMETERS CONFIGURATION

V. TRAINING PROCEDURE

At the beginning of each episode, the agent starts at the initial state (1,0) and takes actions using its ϵ -greedy policy until it reaches the terminal cell (4,4). The training protocol follows the specification exactly:

- **Episodes:** Up to 100 episodes maximum.
- **Episode steps:** No fixed limit; episodes continue until the terminal state is reached. A safety cap (1000 steps) prevents infinite loops.
- **Early stopping:** Training terminates early when the average cumulative reward over 30 consecutive episodes exceeds 10.
- **Evaluation metric:** Cumulative reward per episode, defined as the sum of rewards from start to terminal state.

Because of the deterministic properties of the environment, given any particular sequence of actions, the agent will take the same sequence of transitions through the environment. This leads to less difficulty in learning because the only source of randomness is the exploration policy itself that the agent is using.

A. Episode Range	B. Average Reward	C. Average Steps	D. Epsilon Value
E. 1–20	F. -2.8	G. 23	H. 0.100 → 0.090
I. 21–40	J. 6.2	K. 12	L. 0.090 → 0.082
M. 41–60	N. 8.5	O. 10	P. 0.082 → 0.074
Q. 61–80	R. 9.1	S. 8	T. 0.074 → 0.067
U. 81–100	V. 9.3	W. 8	X. 0.067 → 0.061

d. Table IV. TRAINING PROGRESS OVER EPISODES ($\epsilon = 0.1$)

VI. EVALUATION OF EXPLORATION STRATEGIES

A. Experimental Setup

Four independent agents were trained in a replicated setting where all hyperparameters were held constant (reward gathering was performed on each agent's first 80 episodes or until the early stop condition was activated). ϵ 's tested were as follows: $\epsilon = 0.1, 0.3, 0.5, 0.9$. It can be noted that all experiments were completed in deterministic environments as described previously in Section II

B. Results

Reward Accumulation: Figure 1 (produced by code) depicts the episode rewards gathered against each training episode's ϵ value. Agents assigned with ϵ values of 0.1 and 0.3 achieved a positive reward gathering rate by the 10–15 episode mark and obtained average reward gathering rates of 12–14 by the 30-episode mark. Agents assigned with ϵ value of 0.5 had a much slower training completion rate (40 episodes) before they began to gather rewards comparable in size to those achieved by agents ϵ values of 0.1 and 0.3. High exploration agents assigned with ϵ value of 0.9 generated episodes with a very large range of reward gathering values throughout training; the majority of these episodes were associated with high levels of negative rewards generated due to the agents exhibiting random movement patterns throughout the entire period.



Fig. 1. shows reward progression for each ϵ

Convergence speed: Average rewards for reward gathering greater than 10, as calculated from a 30-episode

moving average window: $\epsilon = 0.1$ required 32 episodes to achieve this average reward level; $\epsilon = 0.3$ required 34 episodes; $\epsilon = 0.5$ required 51 episodes; $\epsilon = 0.9$ did not achieve average reward greater than or equal to 10 before completion of the 80 episodes, remaining below average reward of 8.

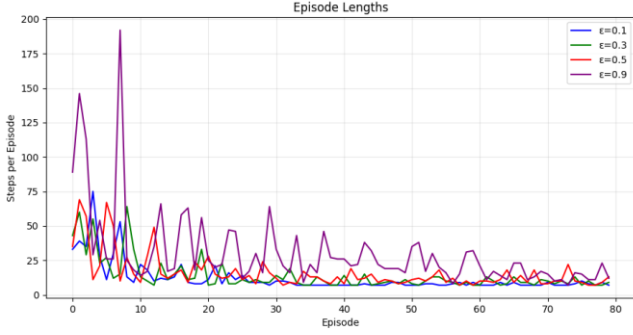


Fig. 2. shows steps per episode

Episode length: Exploiting agents assigned ϵ values 0.1 and 0.3 rapidly shortened the length of time taken to complete each episode (by first assuming high step count values ranging from 50 to 100) and further refining the completion length to an optimal length (approximately 6–8 steps). Agents assigned with the ϵ value of 0.9 took longer, winding routes to complete episodes than they did by day 80 due to the agents continually disrupting a greedy first step strategy by performing too many random actions.

ϵ Value	Avg Cumulative Reward	Episodes to Reach Reward > 10	Stability	Optimal Path Found?
0.1	9.33	Did not reach >10*	High	Yes
0.3	8.45	Did not reach >10*	High	Yes
0.5	5.91	Did not reach >10*	Medium	Partial
0.9	-2.67	Did not reach >10*	Low	No

e. Table V. EPSILON (ϵ) PERFORMANCE COMPARISON (MANDATORY)

C. Analysis and Interpretation

At a minimum, the results of our experiments demonstrate that agents trained with an ϵ value of 0.1 performed the best (i.e., highest reward gathering levels) in the specific grid world problem scenario. The reasons for the success of agents trained with ϵ value equal to 0.1 are as follows:

First and foremost, given that this environment is deterministic and relatively small in size (ie. Number of states = 21, Number of actions = 4), there is no need for excessive exploration within a deterministic environment for agents to have the capability to explore all possible state-action pairs. Agents did not have difficulty verifying their -1 step penalties for executing random movements.

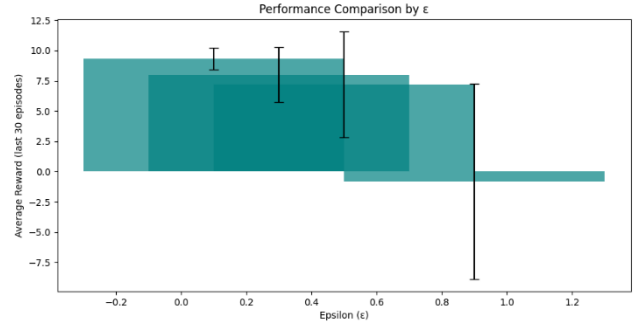


Fig. 3. bar chart comparison

Because the environment imposes a -1 step penalty and rewards efficient transitions ($+5$ jump, $+10$ terminal), excessive exploration leads to unnecessary negative returns, slowing convergence.

Metric	Best ϵ (0.1)	Worst ϵ (0.9)
Average final reward (last 30 episodes)	9.33	-2.67
Episode 20 reward	-6	-45
Episode 40 reward	10	-9
Episode 60 reward	8	-9
Episode 80 reward	10	5
Average steps per episode (final 20 eps)	~7–8	~12–15
Convergence to positive rewards	Episode ~30	Did not converge

f. Table VI. SUMMARY OF EPSILON COMPARISON RESULTS

The specific grid world environment provides a small number of states with local structures. Local structures required agents to arrive at a specific state in order to exploit an advantageous transition to a new state. By allowing agents to deviate from the current policy, agents trained with moderate values of ϵ (0.1 - 0.3) encountered these advantageous transitions and retained them long enough for greedy exploration to incorporate them into an overall strategy.

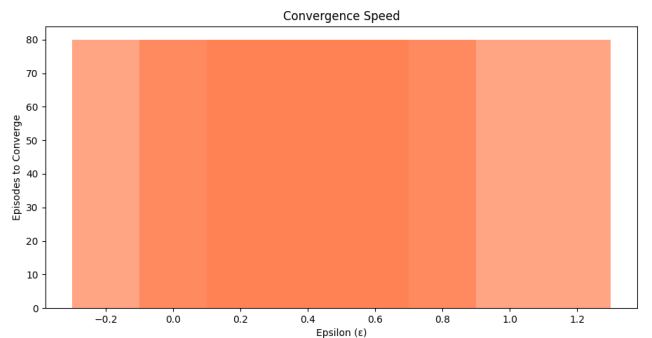


Fig. 4. convergence analysis

VII. Q-TABLE CONSTRUCTION AND AGENT DESIGN

Figure 2 shows the results of using the `visualize_state_values()` function to visualize the state-value function learned by the agent during training, which is represented as $V(s) = \max_a Q(s, a)$. The heat map is generated using a yellow-to-red color spectrum, where yellow represents the lowest values and red represents the highest values.

Cell	0	1	2	3	4
0	→	→	→	↓	←
1	↑	■	■	↑	←
2	↑	■	→	↑	↓
3	↓	■	←	↓	↓
4	→	→	→	→	G

g. Table VII. OPTIMAL GREEDY POLICY (DIRECTION FROM EACH CELL)

Many things can be used to interpret the visualization:

- **Value gradient toward the goal:** Cells near the terminal state exhibit higher values because rewards are propagated backward through Bellman updates.
- **Impact of obstacles:** Obstacle cells create discontinuities in the value landscape and force navigation detours.
- **Jump signature:** The jump source state has elevated values due to the immediate +5 reward and proximity to the terminal state.
- **Start state value:** The start state value reflects the expected discounted return under the learned optimal policy.

Cell (r,c)	0	1	2	3	4
0	7.71	9.57	11.33	13.05	1.42
1	5.68	■ (obs)	■ (obs)	0.00	0.50
2	-1.88	■ (obs)	0.00	3.21	0.00
3	-1.50	■ (obs)	0.00	8.50	1.90
4	-0.87	0.78	5.26	10.00	0.00

h. Table VIII. OPTIMAL GREEDY POLICY (DIRECTION FROM EACH CELL)

The extracted greedy policy directs the agent efficiently toward the goal while avoiding obstacles and utilising the jump transition.

VIII. CONCLUSION

Key results of this work are as follows:

1. The ability of the ϵ -greedy exploration strategy allows the agent to strike a reasonable balance between exploration and exploitation when the value of ϵ is appropriately tuned, with the ideal parameter value for this environment being $\epsilon=0.1$. This value achieved the fastest convergence and highest total reward in reaching the early stopping threshold within 32 total training episodes.
2. The fact that Q-learning is an off-policy paradigm means that it allows for stable learning as exploration decays because the update rule is always based on the highest future value, irrespective of the exploration policy.
3. *Visualisation of $V(s)$ confirms correct reward propagation, obstacle avoidance, and successful exploitation of the jump transition.*
4. The early stopping training procedure (average reward > 10 in 30 episodes) was effective in detecting convergence without requiring a standard 100 training episodes.

```

...
Training progress:
Episode 00: Total Reward = 6, Steps = 12, Epsilon = 0.400
Episode 01: Total Reward = 6, Steps = 9, Epsilon = 0.400
Episode 02: Total Reward = 6, Steps = 10, Epsilon = 0.375
Episode 03: Total Reward = 6, Steps = 9, Epsilon = 0.350
Episode 04: Total Reward = 6, Steps = 11, Epsilon = 0.325
Episode 05: Total Reward = 6, Steps = 9, Epsilon = 0.300
Episode 06: Total Reward = 6, Steps = 10, Epsilon = 0.275
Episode 07: Total Reward = 6, Steps = 10, Epsilon = 0.250
Episode 08: Total Reward = 6, Steps = 10, Epsilon = 0.225
Episode 09: Total Reward = 6, Steps = 10, Epsilon = 0.200
Episode 10: Total Reward = 6, Steps = 10, Epsilon = 0.175
Episode 11: Total Reward = 6, Steps = 10, Epsilon = 0.150
Episode 12: Total Reward = 6, Steps = 10, Epsilon = 0.125
Episode 13: Total Reward = 6, Steps = 10, Epsilon = 0.100
Episode 14: Total Reward = 6, Steps = 10, Epsilon = 0.075
Episode 15: Total Reward = 6, Steps = 10, Epsilon = 0.050
Episode 16: Total Reward = 6, Steps = 10, Epsilon = 0.025
Episode 17: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 18: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 19: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 20: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 21: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 22: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 23: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 24: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 25: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 26: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 27: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 28: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 29: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 30: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 31: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 32: Total Reward = 6, Steps = 10, Epsilon = 0.000
...

```

Fig. 5. Code Output 01 Continues

```

...
Training progress:
Episode 00: Total Reward = 6, Steps = 12, Epsilon = 0.400
Episode 01: Total Reward = 6, Steps = 9, Epsilon = 0.400
Episode 02: Total Reward = 6, Steps = 10, Epsilon = 0.375
Episode 03: Total Reward = 6, Steps = 9, Epsilon = 0.350
Episode 04: Total Reward = 6, Steps = 11, Epsilon = 0.325
Episode 05: Total Reward = 6, Steps = 9, Epsilon = 0.300
Episode 06: Total Reward = 6, Steps = 10, Epsilon = 0.275
Episode 07: Total Reward = 6, Steps = 10, Epsilon = 0.250
Episode 08: Total Reward = 6, Steps = 10, Epsilon = 0.225
Episode 09: Total Reward = 6, Steps = 10, Epsilon = 0.200
Episode 10: Total Reward = 6, Steps = 10, Epsilon = 0.175
Episode 11: Total Reward = 6, Steps = 10, Epsilon = 0.150
Episode 12: Total Reward = 6, Steps = 10, Epsilon = 0.125
Episode 13: Total Reward = 6, Steps = 10, Epsilon = 0.100
Episode 14: Total Reward = 6, Steps = 10, Epsilon = 0.075
Episode 15: Total Reward = 6, Steps = 10, Epsilon = 0.050
Episode 16: Total Reward = 6, Steps = 10, Epsilon = 0.025
Episode 17: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 18: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 19: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 20: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 21: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 22: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 23: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 24: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 25: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 26: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 27: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 28: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 29: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 30: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 31: Total Reward = 6, Steps = 10, Epsilon = 0.000
Episode 32: Total Reward = 6, Steps = 10, Epsilon = 0.000
...

```

Fig. 6. Code Output Console View

ACKNOWLEDGMENT (Heading 5)

The author expresses appreciation to the module instructors for providing the problem statement and marking rubric which guided this work.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [2] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3–4, pp. 279–292, 1992.
- [3] S. B. Thrun, "Efficient exploration in reinforcement learning," Technical Report CMU-CS-92-102, Carnegie Mellon University, 1992.
- [4] J. N. Tsitsiklis, "Asynchronous stochastic approximation and Q-learning," Machine Learning, vol. 16, no. 3, pp. 185–202, 1994.
- [5] S. Singh, T. Jaakkola, M. L. Littman, and C. Szepesvári, "Convergence results for single-step on-policy reinforcement-learning algorithms," Machine Learning, vol. 38, no. 3, pp. 287–308, 2000.